Analytical Report

Analysis of Corporate Students' Engagement with the IT Resume Platform

1. Purpose of the Study

IT Resume is an educational IT platform where students from Company 1 solve problems, take tests, and use the internal currency, CodeCoins.

The purpose of this study was to understand how students interact with the platform, how actively they progress through the curriculum, where the main difficulties arise, and how actively they use the platform's internal economy.

At the beginning of the study, it was not known at which stage the main problem occurred. A student might register but never start solving problems, complete only a small part of the available curriculum, encounter many failed attempts, or not use certain platform features at all.

The study was therefore organized around a series of questions:

1. How actively do students use the platform?
2. How far do they get when working on problems?
3. How much progress do they make through the curriculum, individually and as a cohort?
4. Where exactly do students encounter difficulties solving problems?
5. Do students use tests as an additional learning tool?
6. How actively is the internal CodeCoins currency used?

This approach provides an overall picture of student behavior first, then examines individual stages of the learning process and potential opportunities for improvement.

2. Data and Analytical Considerations

The analysis used data on Company 1 students, platform logins, code runs and submissions, problem-solving results, tests, and CodeCoins transactions.

Different events were used as indicators of activity when calculating different metrics. In particular, platform logins were recorded through `userentry`, work on problems through `coderun` and `codesubmit`, and test activity through `teststart` and `testresult`.

The analysis also distinguished between problems assigned to Company 1 and other problems on the platform. This proved to be an important limitation of the initial assumption: students worked on problems beyond those assigned to their company.

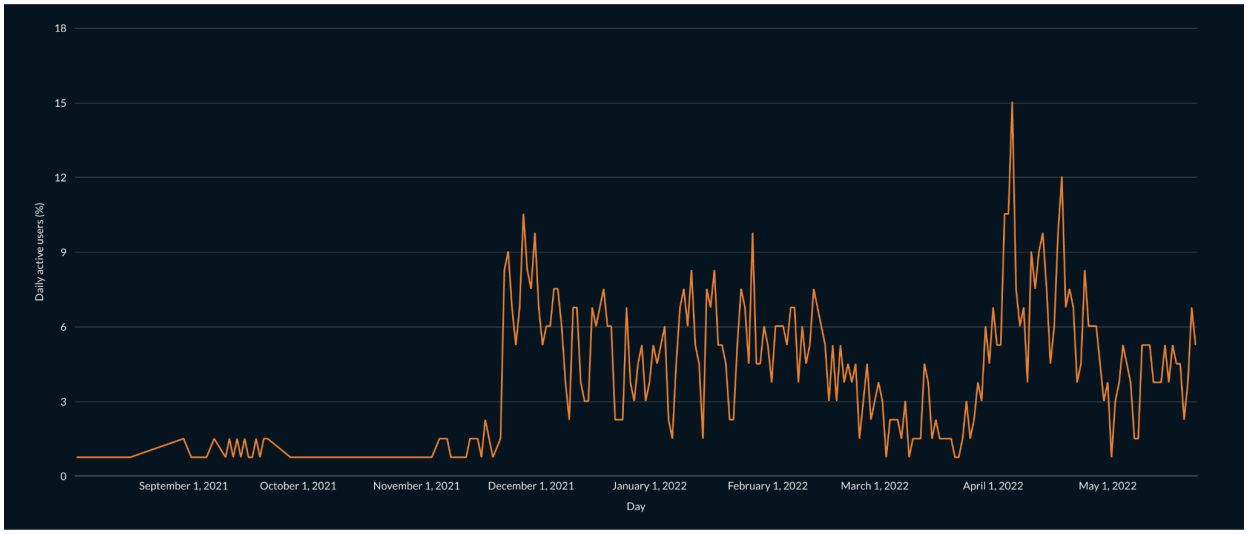
Of the 124 problems assigned to Company 1, students ran code for 121, submitted solutions for 122, and successfully solved 122. Thus, at the cohort level, at least one student engaged with most of the assigned curriculum.

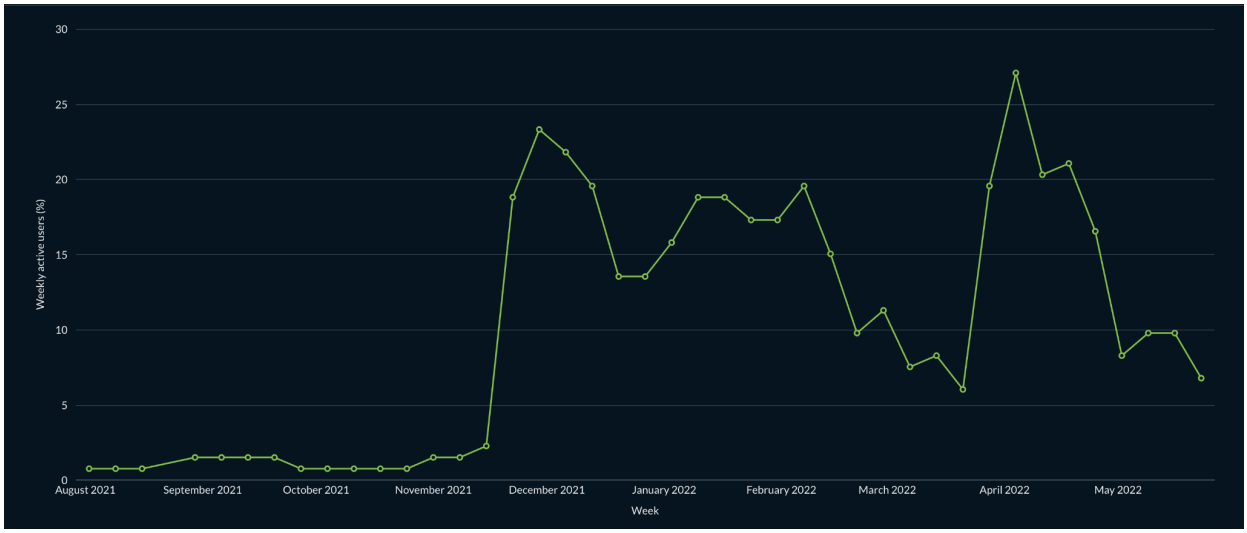
However, these data show collective coverage of problems by the cohort, not each student's individual completion of the entire curriculum. These metrics therefore cannot be interpreted as the share of students who completed the program.

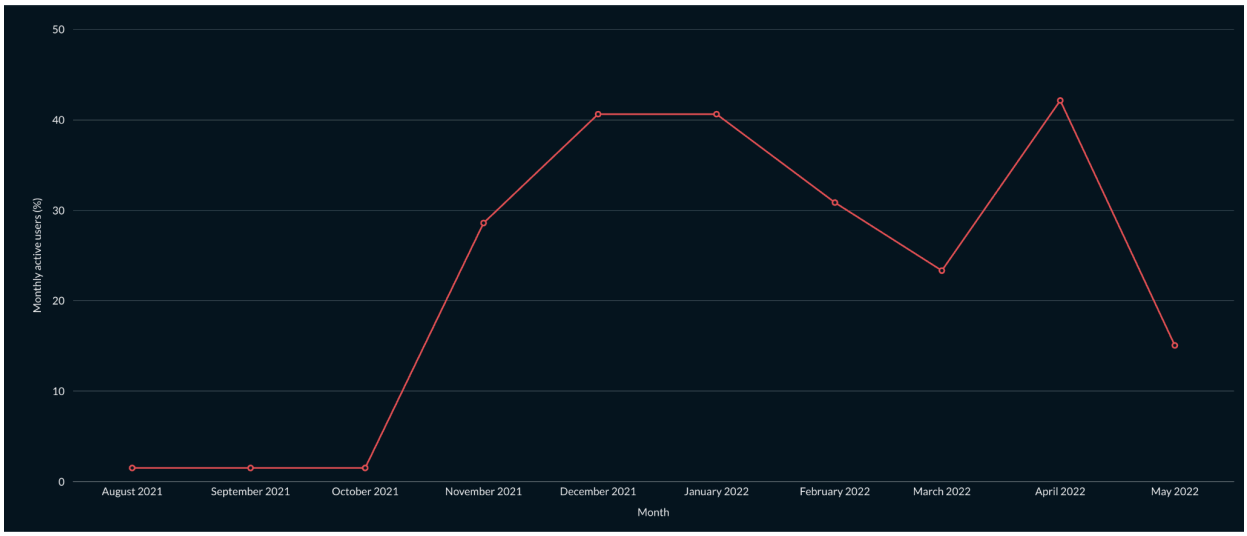
3. How Engaged Are Students in Learning?

The first step was to assess overall student activity. If a substantial share of students rarely interacts with the platform, a subsequent decline in the number of solved problems may reflect the fact that students never start learning, rather than the difficulty of the material.

To assess this, I examined the share of students who logged in to the platform daily, weekly, and monthly.







DAU was the lowest of these metrics, while MAU was considerably higher. The charts also show substantial fluctuations in activity over time: monthly activity reached approximately 40%, while daily activity was much lower.

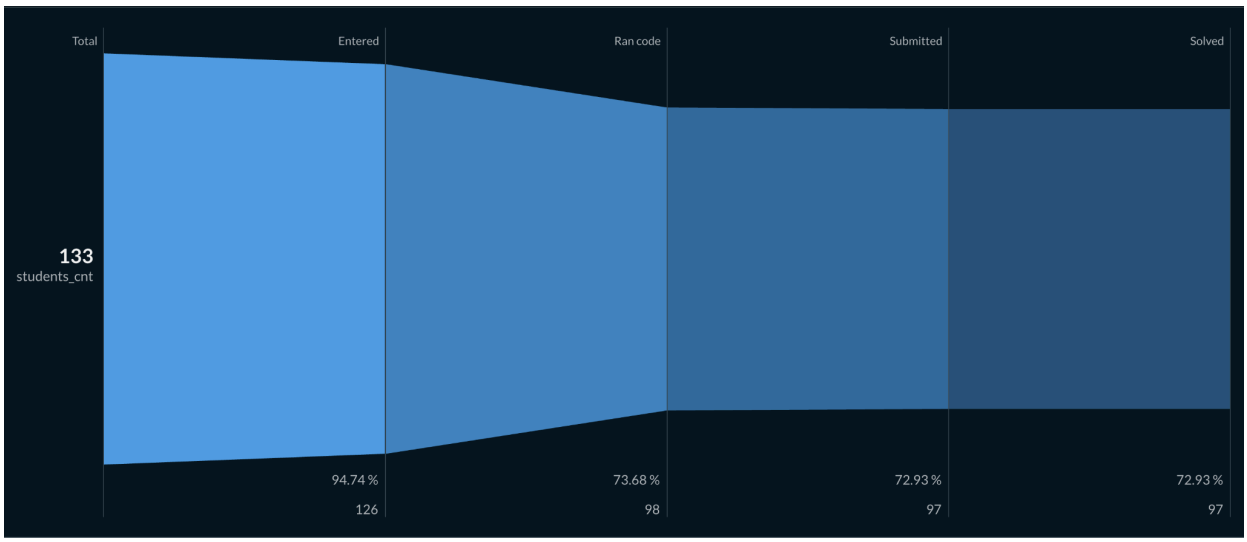
Overall, students use the platform irregularly and appear to engage with it periodically rather than every day.

However, logging in alone does not mean that a student is actually studying. The next question was therefore: **what happens after a student logs in?**

4. What Happens After Students Log In?

To examine the student journey in more detail, I built an overall funnel:

Registered → Logged in to the platform → Ran code → Submitted a solution → Solved a problem successfully.



Of the 133 students from Company 1, 126 logged in to the platform, 98 ran code, 97 submitted a solution, and 97 achieved at least one successful solution.

The most noticeable drop occurs between logging in and the first code run. Once students begin working directly on a problem, they are much more likely to submit a solution and achieve a successful result.

This raises the first important question for further analysis:

Is the problem that students do not start learning, or that they find it difficult to continue once they have started?

To answer this question, I also examined the number of days on which students showed learning activity.

5. Do Students Return to Learning Regularly?

An active learning day was defined as a day on which a student performed at least one meaningful learning action: running code, submitting a solution, or starting a test.

After combining these actions, the median number of active learning days was **3 days**.

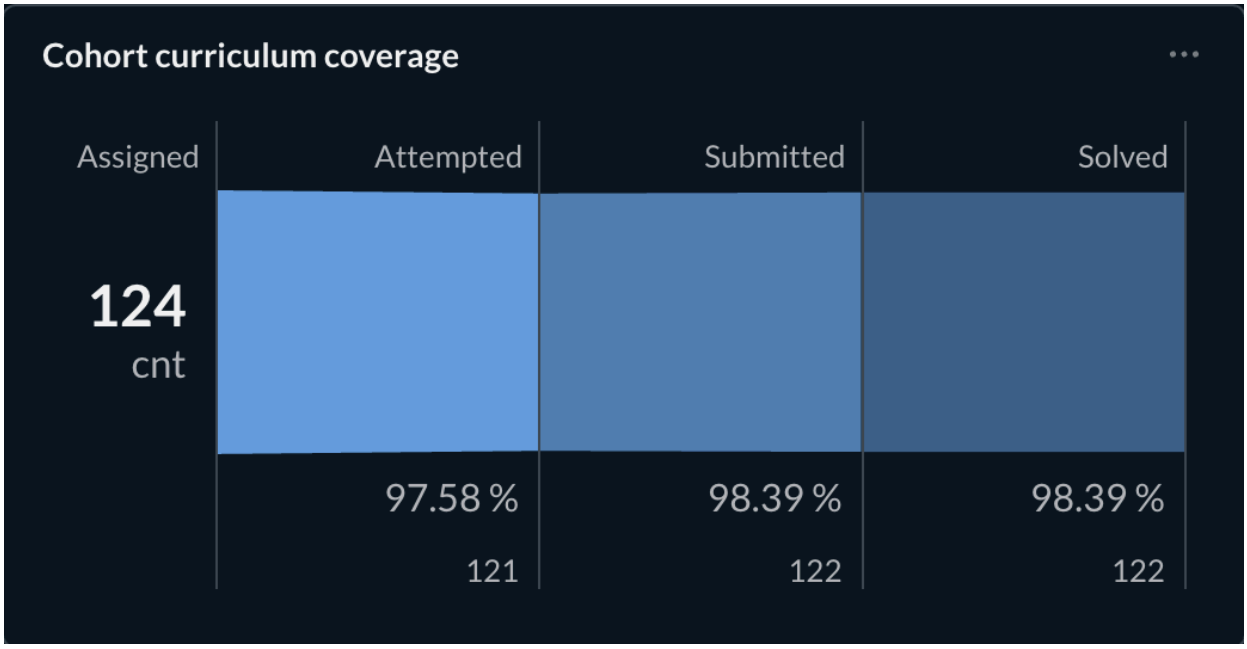
About a quarter of students had no recorded active learning days at all, while a small group of students showed substantially higher activity.

This suggests that low engagement is not driven only by a few individual students. A substantial part of the cohort had limited interaction with learning content, although a small segment was very active.

This leads to the next question: **if a student does start solving problems, how far do they progress?**

6. How Far Do Students Progress Through the Curriculum?

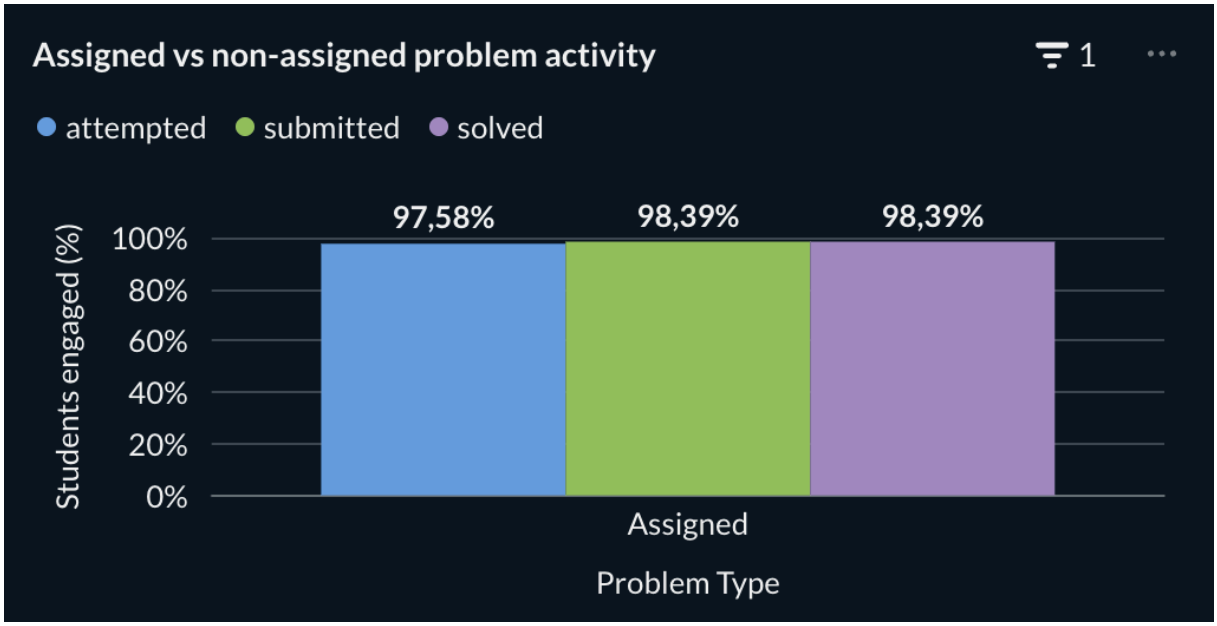
At the cohort level, the result was relatively high.



Of the 124 problems assigned to Company 1, at least one student ran code for approximately 97.6% of them, submitted a solution for 98.4%, and successfully solved approximately 98.4%.

In other words, when students are viewed as a single group, almost the entire assigned curriculum was reached.

However, the result looks much less optimistic when we move from the cohort to individual students.



Median individual progress was **5.65%**, while the highest result was approximately **43.55%**.

This reveals an interesting gap:

The cohort as a whole covers almost the entire curriculum, but an individual student usually solves only a small part of it.

This means that high collective coverage should not be interpreted as high individual progress. Rather, different students appear to solve different parts of the overall curriculum.

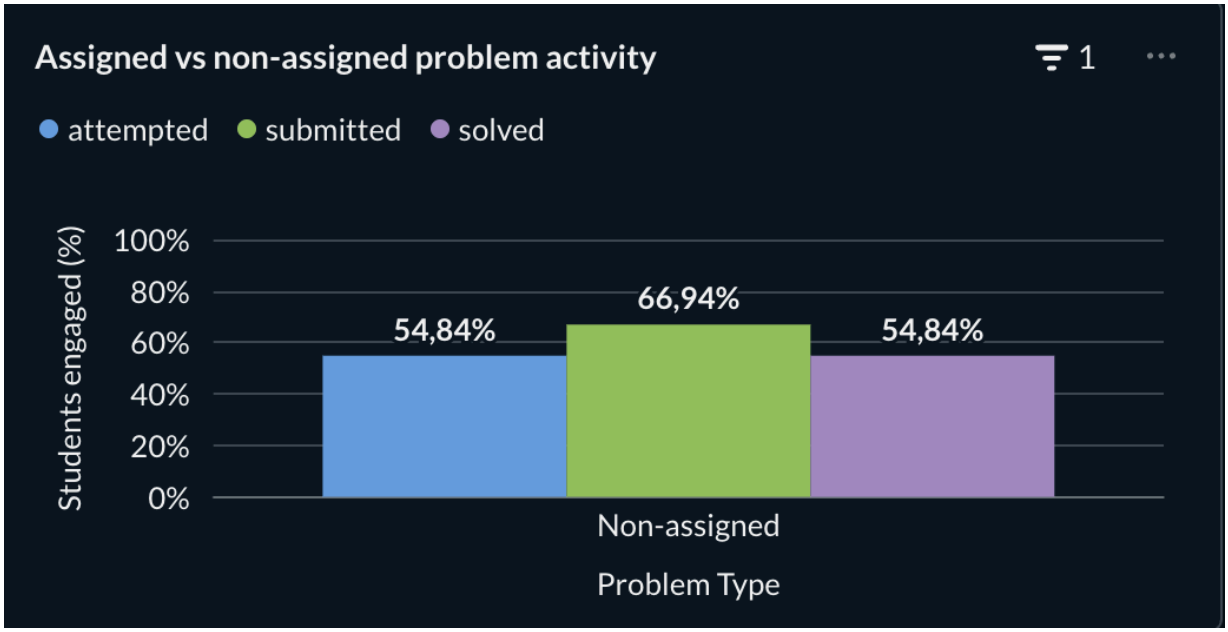
To investigate this observation, I also compared student activity on assigned and non-assigned problems.

7. Do Students Solve Only the Problems Assigned to Them?

The analysis showed that students from Company 1 work not only on problems assigned as part of their program.

For assigned problems, coverage was around 98% across the main stages. For non-assigned problems, students also engaged with more than half of the available problem set.

Notably, non-assigned problems were submitted for evaluation more often than they were ultimately solved successfully: about **66.9%** of these problems had submissions, while approximately **54.8%** were successfully solved.



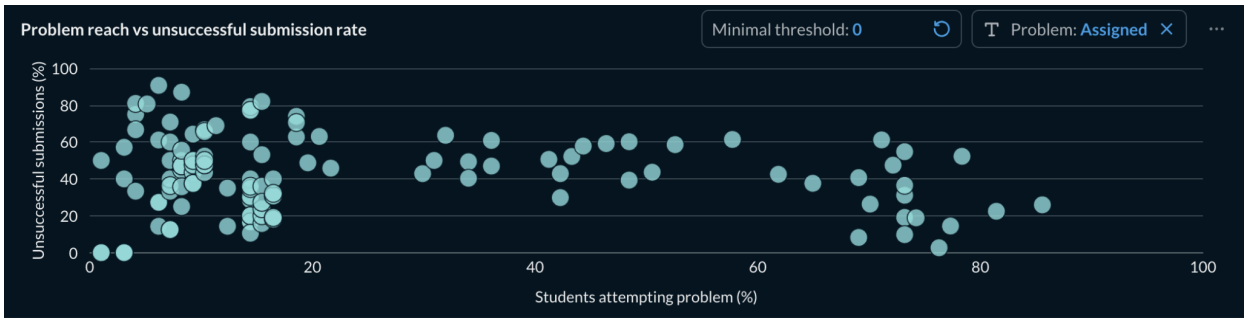
This differs from the pattern for assigned problems, where nearly all problems students engaged with were eventually solved successfully.

This raises the next question:

When students start solving problems, which ones cause them the most difficulty?

8. Where Do the Main Problem-Solving Difficulties Occur?

To analyze friction, I considered two characteristics together: the share of students who attempted a problem and the share of submissions that were unsuccessful.



Most problems were attempted by a relatively small share of students. Among the problems students did attempt, however, unsuccessful submissions often accounted for approximately **20–50%**.

This suggests that the issue is not simply a lack of interest in particular problems. Students who do start working on them often encounter errors before reaching a correct result.

Further analysis shows that errors do not necessarily lead students to abandon a problem. The median number of failed submissions before the first successful solution was **2**.

Thus, a typical student encounters several unsuccessful attempts but eventually reaches a correct solution.

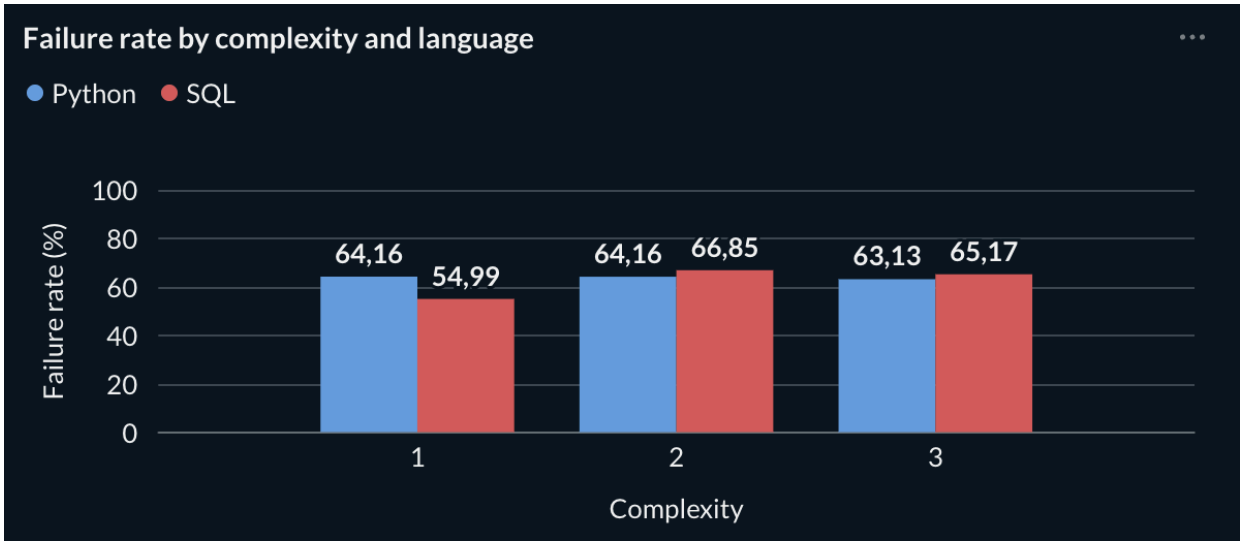
This leads to the next question: **are these difficulties related to the objective difficulty of a problem or to the programming language?**

9. Does Problem-Solving Difficulty Depend on Problem Level and Language?

I compared unsuccessful submission rates across problem difficulty levels and programming languages.

A direct relationship between language and difficulty cannot be inferred because each problem can be solved in only one language. It is therefore more appropriate to compare languages within the same difficulty level.

The resulting values were relatively similar.



For difficulty level 1, the unsuccessful submission rate was approximately **64% for Python and 55% for SQL**. At level 2, the rates were approximately **64% and 67%**; at level 3, they were approximately **63% and 65%**, respectively.

Overall, the data show no pronounced difference between languages within the same difficulty level.

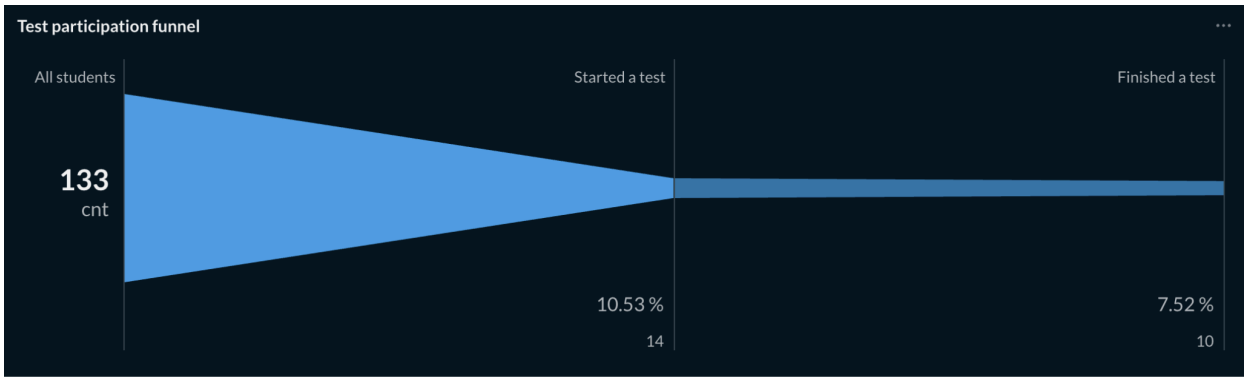
Nor is there an expected monotonic increase in the error rate from difficulty level 1 to level 3.

This means that language or the nominal difficulty level alone does not provide a sufficient explanation. Further investigation would need to consider the characteristics of individual problems and student behavior at the problem level.

10. Do Students Use Tests Enough to Check Their Knowledge?

After analyzing problem-solving activity, I examined tests as a second major form of learning activity.

The results were noticeably weaker than those for problem-solving.



About **10% of students started at least one test**, while approximately **7.5% completed one**. More than 84% of students did not start any tests at all.

Even among students who use this section, most completed only 1 to 5 of the 24 available tests.

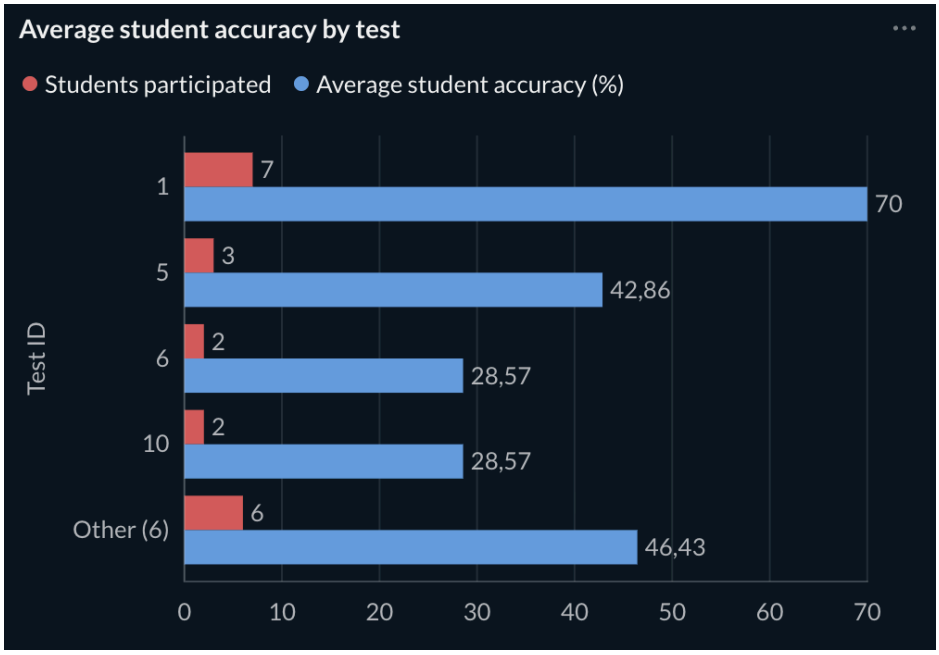
Thus, tests are used substantially less actively than problem-solving tasks.

However, a low number of completed tests does not by itself indicate the quality of students' knowledge. The next question was therefore: **how successfully do students perform on the tests they do take?**

11. How Well Do Students Perform on Tests?

Average answer accuracy across tests was relatively low.

The most popular test reached approximately **70% average accuracy**, while the values for the following tests were noticeably lower.



Tests therefore have two notable characteristics: only a small share of students reach them, and among those who do, the proportion of correct answers is not always high.

This suggests that tests may be a useful area for further investigation into learning quality. However, the current analysis cannot determine whether low accuracy is caused by test difficulty, insufficient preparation, or low student engagement.

12. How Do Students Use the Platform's Internal Economy?

The final area I examined was CodeCoins.

Almost all students from Company 1 received the internal currency at least once: the share was **94.7%**.

However, only **69.2%** of students spent CodeCoins.

The median amount of CodeCoins earned was **203**, while the median amount spent was considerably lower, at **57.5**.

Thus, earning CodeCoins is much more widespread than spending them, and a typical student retains some of their accumulated currency.

13. What Do Students Spend CodeCoins On?

CodeCoins earnings vary considerably from month to month. Some months show pronounced peaks, driven primarily by specific types of actions rather than by a uniform increase across all activity types.

CodeCoins spending is much lower in volume. Most spending goes toward purchasing problems, while hints, tests, and ready-made solutions account for considerably smaller shares.

Thus, students do use the platform's internal economy, but the main use of CodeCoins is to gain access to additional problems.

However, the available data do not establish that CodeCoins directly increase student engagement. This would require a separate analysis of the relationship between currency use and subsequent learning behavior.

14. How Do the Findings Fit Together?

When the individual metrics are considered together, they form a fairly coherent picture.

The main issue appears to occur before problem-solving begins, rather than within the problem-solving process itself.

A substantial share of students show learning activity infrequently, and the most noticeable drop in the overall funnel occurs between logging in and starting to work with code.

Once students start solving problems, the picture is considerably better: students who engage with problems often reach a successful result. Even several unsuccessful attempts usually do not prevent them from eventually solving a problem.

At the same time, there is an interesting gap between collective and individual progress. The cohort covers almost all assigned problems, yet a typical individual student solves only a small part of the curriculum.

Tests are used much less than problem-solving tasks. Almost all students earn CodeCoins, but they spend them much less often, primarily on additional problems.

The findings can therefore be summarized as the following sequence:

Log in to the platform → Begin meaningful work → Continue learning → Achieve a result.

The main opportunity for improvement may not simply be to increase the number of logins, but to shorten the gap between logging in and taking a meaningful learning action.

15. What Could Be Investigated Next?

Based on this analysis, I would identify several areas for further investigation.

First, the stage between logging in and starting a problem should be examined separately, since this is where the most noticeable drop in the funnel occurs.

Second, it would be useful to investigate why individual progress remains low despite almost complete collective curriculum coverage. Students may be completing different parts of the curriculum, or some problems may fall outside their individual learning paths.

Third, individual problems with high unsuccessful submission rates should be investigated. Problems attempted by a relatively large number of students but requiring several attempts to solve successfully are particularly interesting.

Finally, the use of tests and CodeCoins could be explored further, not only in terms of how many students use these features, but also whether their use is associated with subsequent learning activity.

16. Limitations of the Study

Several limitations should be considered when interpreting the results.

First, different platform events are recorded in different tables, so the activity metrics depend on which actions are treated as learning activity.

Second, logging in and meaningful learning activity are not the same thing. A login does not mean that a student solved problems or took tests.

Third, collective curriculum coverage cannot be interpreted as individual curriculum completion: the same cohort-level result can be achieved by many students, each of whom solves only a small part of the problem set.

Fourth, comparisons between programming languages are limited because each problem is available in only one language. It is therefore impossible to separate the effect of the language from the characteristics of the specific problems.

Finally, this is a descriptive study. The patterns identified describe student behavior but do not, by themselves, prove cause-and-effect relationships.

17. Conclusion

The study began with a broad question: how do students from Company 1 interact with IT Resume, and where do the main limitations in their learning journey occur?

The analysis indicates that the main issue is low and irregular engagement. A substantial share of students rarely perform learning activities, and the most noticeable drop occurs between logging in to the platform and starting work on a problem.

Once students begin solving problems, outcomes improve considerably. At the cohort level, almost the entire assigned curriculum was reached, and most problems that students engaged with eventually received a successful solution.

At the same time, individual progress remains much lower than collective progress: a typical student solves only a small part of the assigned curriculum.

Tests are used much less often than problem-solving tasks. Students frequently earn CodeCoins but spend them considerably less often, mainly on additional problems.

Thus, the most promising direction for further analysis is the journey from logging in to the first meaningful learning action, as well as the reasons for low individual progress despite high collective curriculum coverage.